SMART CHILD SAFETY SYSTEM USING ESP32

REVIEW – 2 REPORT

Manan Gupta 23BCE0930

Aryan Khan 23BCB0115

INTRODUCTION

The Smart Child Safety System is an embedded IoT-based solution developed to enhance the safety and monitoring of children through real-time location tracking and intelligent alert mechanisms. With increasing concerns regarding child safety in urban and semi-urban environments, there is a growing need for systems that can continuously monitor location and respond immediately to unsafe situations.

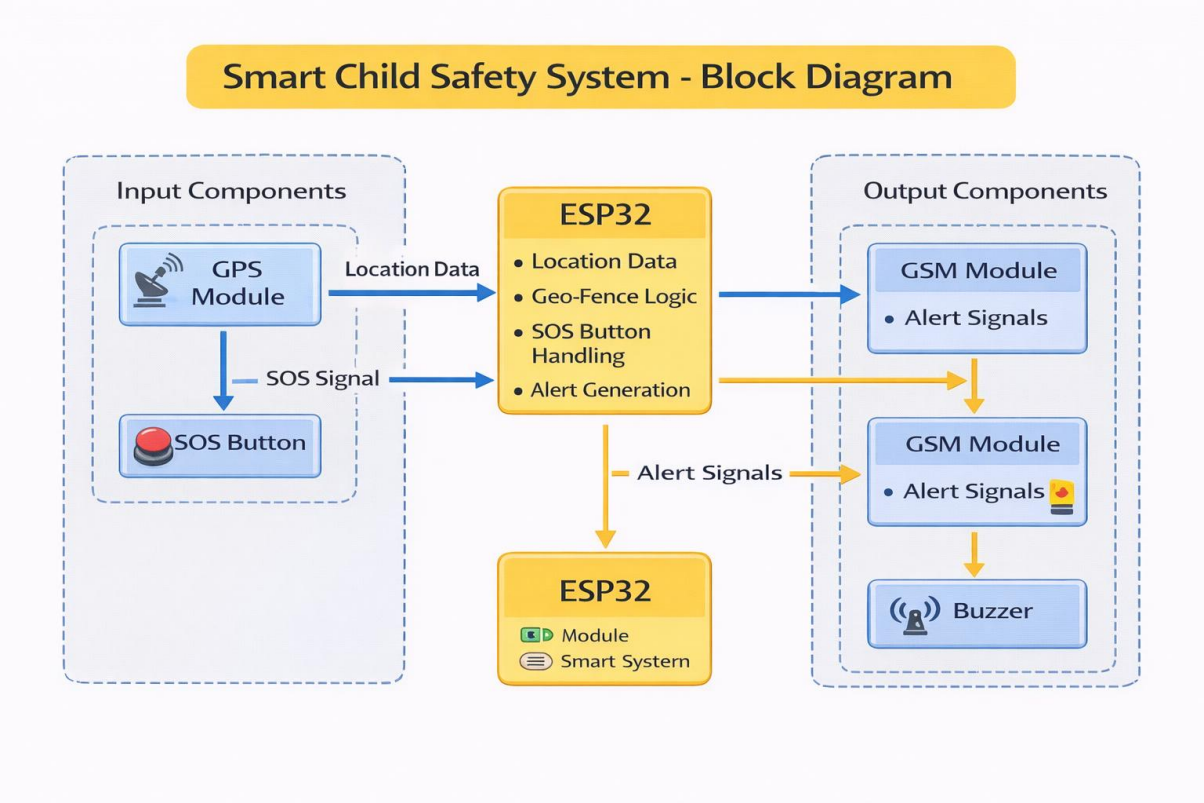
This project utilizes the ESP32 microcontroller as the central processing unit, integrated with a GPS module for location acquisition and a GSM module for communication. The system is designed to not only track the child's position but also analyze it using geo-fencing logic, thereby enabling automatic detection of unsafe conditions. In addition to automated alerts, a manual SOS feature is incorporated to allow immediate emergency signaling.

Unlike basic tracking systems that only provide location data, this system introduces decision-making capability by combining spatial (location-based) and temporal (time-based) conditions. The system continuously evaluates whether the child is within a predefined safe zone and triggers alerts if a boundary violation occurs during critical time intervals. Furthermore, the implementation includes mechanisms to prevent redundant alerts and ensure efficient communication.

The objective of this review is to present the detailed architecture, component interfacing, modeling techniques, data and control flow analysis, optimization strategies, and real-time considerations involved in the system design. This stage of the project focuses on transforming the conceptual model into a structured and partially implemented system, covering both hardware and software aspects in detail.

The proposed system demonstrates the integration of embedded systems, real-time processing, and communication technologies to create a reliable and responsive child safety solution.

1. ARCHITECTURE / BLOCK DIAGRAM



System Overview

The Smart Child Safety System is an embedded system designed to monitor a child's location in real time and ensure safety using geo-fencing and emergency alert mechanisms. The system integrates sensing, processing, and communication components to create a reactive and reliable safety solution.

The architecture follows a modular approach consisting of input, processing, and output units. Each unit performs a specific function and contributes to the overall system operation.

Functional Description

Input Layer

The input layer consists of:

- GPS module for continuous location tracking
- SOS button for manual emergency triggering

The GPS module provides real-time latitude and longitude values, while the SOS button generates an interrupt signal when pressed.

Processing Layer

The ESP32 microcontroller acts as the central processing unit. It performs the following functions:

- Reads GPS data using UART
- Executes geo-fencing logic
- Processes SOS signals using interrupt handling
- Controls output devices

The decision-making capability is implemented in software, where the system determines whether the child is inside or outside a predefined safe zone.

Output Layer

The output layer consists of:

- GSM module for sending SMS alerts
- Buzzer for generating local alerts

When a safety violation or emergency condition is detected, the system triggers both remote and local alerts.

Data Flow

GPS → ESP32 → Distance Calculation → Decision Logic → Alert System

---

2. COMPONENT ANALYSIS

Component	Function	Interface
ESP32	**Central processing, UART/GPIO handling**	**UART (GPS/GSM), GPIO 4 (button), GPIO 5 (buzzer) SMART-CHILD-SAFETY-SYSTEM-USING-ESP32.docx**

Component	Function	Interface
GPS Module	**Location via NMEA sentences, trilateration**	**UART (RX/TX) SMART-CHILD-SAFETY-SYSTEM-USING-ESP32.docx**
GSM Module	**SMS with Google Maps links**	**UART (GPIO 16/17), AT commands SMART-CHILD-SAFETY-SYSTEM-USING-ESP32.docx**
SOS Button	**Manual emergency trigger**	**Digital input (GPIO 4, pull-up) SMART-CHILD-SAFETY-SYSTEM-USING-ESP32.docx**
Buzzer	**Local sound alert**	**Digital output (GPIO 5) SMART-CHILD-SAFETY-SYSTEM-USING-ESP32.docx**

2.1 ESP32 MICROCONTROLLER

Working Principle

The ESP32 is a 32-bit dual-core microcontroller that executes instructions stored in program memory. It processes input signals from peripherals, performs computations, and generates output signals based on programmed logic.

Interfacing

The ESP32 interfaces with external components using:

- UART communication for GPS and GSM modules
- GPIO pins for button input and buzzer output

On-Chip Peripherals

The following peripherals are utilized:

- UART: For serial communication
- GPIO: For digital input/output

- Timers: For time-based operations
- Interrupt controller: For handling SOS button

Optimization

Efficient utilization of peripherals reduces processing delay and improves system responsiveness. Hardware UART is preferred over software serial communication for faster data transfer.

UPDATED CODE: C++

```
#include <Arduino.h>

// PINS
#define BUTTON_PIN 4
#define BUZZER 5

// Geo-fence (School)
float schoolLat = 12.970972;
float schoolLng = 79.164083;
float radius = 0.001;

// Simulated GPS values
float currentLat = 12.970972;
float currentLng = 79.164083;

unsigned long lastAlertTime = 0;

// Button state (for de-bounce)
bool buttonPressed = false;
```

```
//  BUZZER FUNCTION
void triggerBuzzer() {
  digitalWrite(BUZZER, HIGH);
  delay(1000);
  digitalWrite(BUZZER, LOW);
}

// SMS SIMULATION
void sendSMS(String message) {
  Serial.println("SMS SENT:");
  Serial.println(message);
  Serial.println("------------------------------------------------------------");
}

//  ALERT
void sendAlert() {
  String msg = "ALERT! Child left SAFE ZONE\n";
  msg += "https://maps.google.com/?q=";
  msg += String(currentLat, 6) + "," + String(currentLng, 6);

  sendSMS(msg);
  triggerBuzzer();
}

// SOS
void sendSOS() {
  String msg = "SOS!EMERGENCY! \n";
```

```
  msg += "https://maps.google.com/?q=";
  msg += String(currentLat, 6) + "," + String(currentLng, 6);

  sendSMS(msg);
  triggerBuzzer();
}

// TRACKING
void sendLocation() {
  String msg = "LOCATION:\n";
  msg += "https://maps.google.com/?q=";
  msg += String(currentLat, 6) + "," + String(currentLng, 6);

  sendSMS(msg);
}

// SETUP
void setup() {
  Serial.begin(115200);

  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(BUZZER, OUTPUT);

  Serial.println("****** SMART CHILD SAFETY SYSTEM STARTED ******");
}

//  LOOP
void loop() {
```

```
// SIMULATED GPS (WE WILL CHANGE FOR TESTING)

// Inside safe zone
currentLat = 12.970972;
currentLng = 79.164083;

// Uncomment below to simulate OUTSIDE zone
// currentLat = 12.975000;
// currentLng = 79.170000;

// SOS Button (FIXED - WITHOUT SPAM)
if (digitalRead(BUTTON_PIN) == LOW && !buttonPressed) {
  Serial.println("SOS BUTTON PRESSED");
  sendSOS();
  buttonPressed = true;
  delay(200);  // debounce
}

if (digitalRead(BUTTON_PIN) == HIGH) {
  buttonPressed = false;
}

// Simulated TRACK command
String incomingMsg = "";
// Uncomment to test TRACK
// incomingMsg = "TRACK";
```

```
if (incomingMsg == "TRACK") {
  Serial.println("TRACK REQUEST RECEIVED");
  sendLocation();
}

// Time logic
int hour = 10;
bool schoolTime = (hour >= 8 && hour < 14);

// Distance calculation
float distance = sqrt(
  pow(currentLat - schoolLat, 2) +
  pow(currentLng - schoolLng, 2)
);

bool outside = (distance > radius);

// Geo-fence check (Without SPAM)
if (schoolTime && outside) {
  if (millis() - lastAlertTime > 5000) {
    Serial.println("GEOFENCE VIOLATION!");
    sendAlert();
    lastAlertTime = millis();
  }
} else {
  Serial.println("Child is inside safe zone");
}
```

```
  delay(2000);
}
```

2.2 GPS MODULE

Working Principle

The GPS module determines geographical location using signals received from multiple satellites. It calculates position based on the time delay of signal transmission using trilateration.

Communication Protocol

The GPS module communicates using UART protocol and transmits data in NMEA sentence format.

Data Processing

The ESP32 extracts useful information such as latitude and longitude from the received data stream.

Strategy

A polling approach is used to continuously read GPS data at regular intervals.

2.3 GSM MODULE

Working Principle

The GSM module enables communication over a cellular network. It is used to send SMS alerts containing location information.

Communication Protocol

Communication with the GSM module is performed using AT commands over UART.

Functionality

- Sends emergency alerts
- Responds to tracking commands

2.4 SOS BUTTON

Working Principle

The SOS button is a digital input device that generates a LOW signal when pressed. It is configured using internal pull-up resistance.

Handling Method

The button is handled using an interrupt-based approach to ensure immediate response during emergency situations.

2.5 BUZZER

Working Principle

The buzzer converts electrical signals into sound. It is activated when an alert condition occurs.

Function

- Provides local alert indication
- Enhances system responsiveness

---

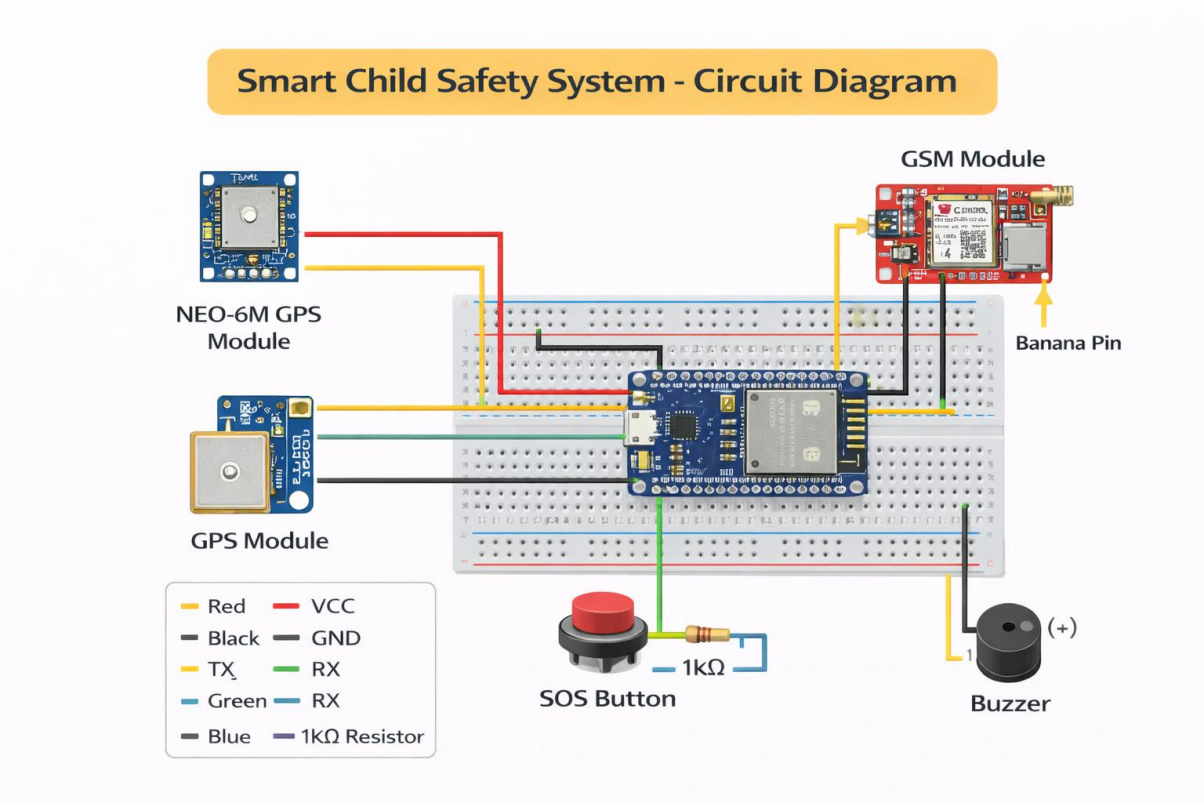
3. INTERFACING DETAILS

Component	Pins Used	Protocol
GPS	RX, TX	UART
GSM	GPIO 16, 17	UART
Button	GPIO 4	Digital Input
Buzzer	GPIO 5	Digital Output

All components share a common ground. The GSM module requires an external power source due to higher current requirements.

---

4. CIRCUIT DIAGRAM



Circuit Description

The GPS module is connected to the ESP32 using UART communication. The GSM module is connected using a separate UART channel. The SOS button is connected to a GPIO pin configured with internal pull-up. The buzzer is connected to a GPIO pin configured as output.

---

5. WORKING PRINCIPLE

The system operates continuously in a loop-based structure.

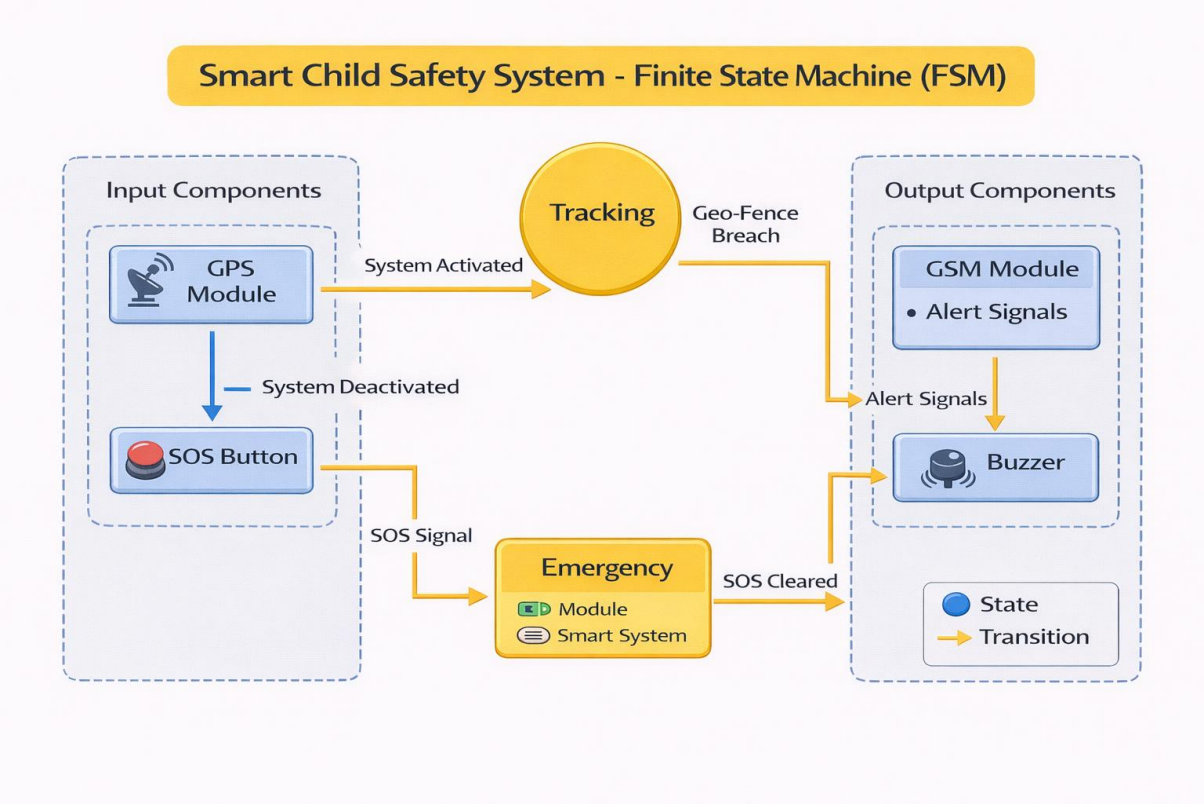
1. GPS module provides location data
2. ESP32 reads and extracts coordinates
3. Distance between current location and safe zone is calculated
4. If the distance exceeds a predefined radius, an alert is triggered
5. If the SOS button is pressed, an emergency alert is sent

6. Alerts include location information and are transmitted via GSM

---

6. MODELING

6.1 FINITE STATE MACHINE



States

- Idle: System initialization state
- Tracking: Continuous monitoring state
- Alert: Triggered when geo-fence is violated
- Emergency: Triggered by SOS button

State Transitions

Current State	Condition	Next State
Idle	System start	Tracking
Tracking	Inside boundary	Tracking

Current State	Condition	Next State
Tracking	Outside boundary	Alert
Tracking	SOS pressed	Emergency
Alert	Condition resolved	Tracking
Emergency	Reset	Tracking

6.2 SEQUENTIAL PROGRAMMING MODEL

The system follows a loop-based sequential model where operations are executed step by step.

Pseudocode

```
Initialize system
Loop:
Read GPS data
Check boundary condition
If violation → trigger alert
Check SOS button
If pressed → trigger emergency
```

6.3 CONCURRENT PROGRAMMING MODEL

The system can be extended using multitasking where different operations are executed simultaneously:

- GPS data acquisition
- Geo-fence evaluation
- Communication handling

6.4 OBJECT-ORIENTED MODEL

The system can be modularized using classes:

- GPSHandler: Manages GPS data
- AlertSystem: Handles alert generation

- GeoFence: Performs boundary checks

6.5 DISCRETE EVENT SYSTEM MODEL

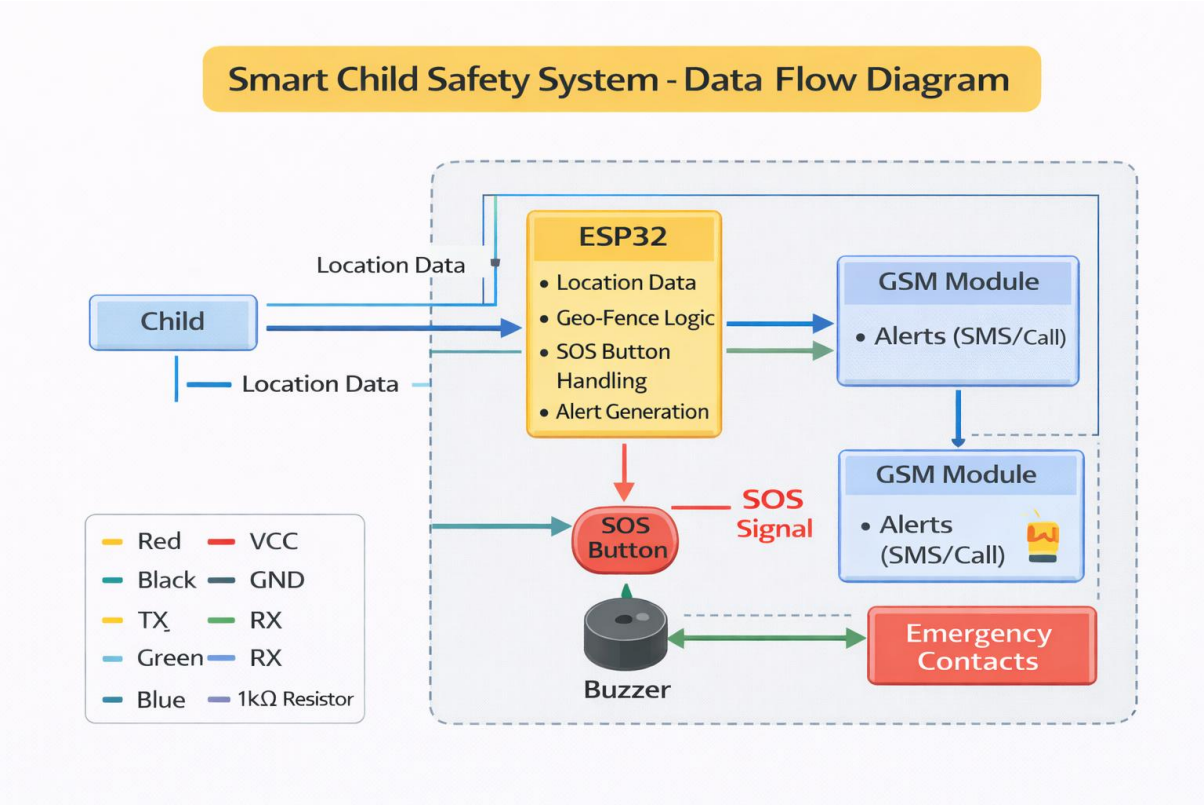
The system responds to events such as:

- GPS updates
- SOS button press
- Geo-fence violation

Race conditions are avoided using proper flag handling and timing control.

---

7. DATA FLOW GRAPH (DFG)



Description

The DFG represents the flow of data through the system.

Variables

- Latitude
- Longitude
- Distance

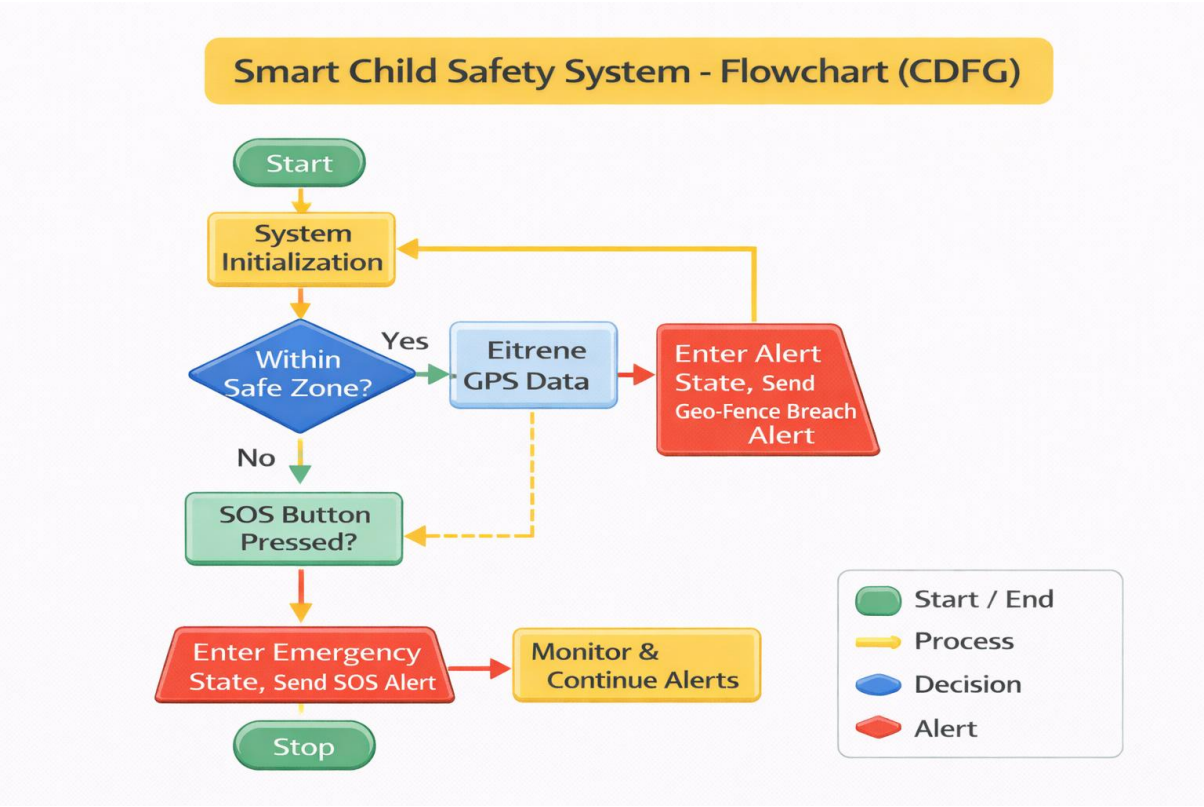
- Alert flag

Characteristics

- Acyclic structure
- Clear data dependencies
- Possibility of parallel execution

---

8. CONTROL DATA FLOW GRAPH (CDFG)



Description

The CDFG represents both control flow and data flow within the system.

Execution Flow

Start → Initialize → Read GPS → Compute distance → Check condition → Trigger alert → Repeat

Worst Case Execution Time

Occurs when:

- GPS data acquisition delay
- GSM transmission delay

9. OPTIMIZATION

Code Optimization

Efficient logic is used to minimize redundant computations and improve execution speed.

Memory Optimization

Variables are reused wherever possible to reduce memory usage.

Energy Optimization

Power consumption is reduced by limiting unnecessary operations and controlling peripheral usage.

Compiler Optimization

Compiler flags such as -O2 and -Os can be used to improve performance and reduce code size.

Profiling

System performance can be analyzed to identify bottlenecks, especially in communication delays.

10. REAL-TIME REQUIREMENTS

Task Classification

Type	Example
Periodic	GPS data acquisition
Aperiodic	SOS button

Type	Example
Sporadic	Geo-fence violation

Timing Constraints

- GPS update interval: 2 seconds
- Alert response time: within 5 seconds

Scheduling

An event-driven scheduling approach is used to handle real-time events efficiently.

Thread Implementation

Tasks can be implemented as threads:

- GPS task
- Alert task
- Communication task

Synchronization mechanisms such as semaphores can be used to avoid conflicts.

---

11. IMPLEMENTATION STATUS

- Hardware setup completed
- GPS and GSM modules interfaced
- Geo-fencing logic implemented
- SOS functionality implemented
- Alert system tested

---

12. CONCLUSION

The Smart Child Safety System demonstrates an effective integration of embedded systems and communication technologies. It provides real-time monitoring and rapid response to emergency situations through automated and manual alert mechanisms.

13. FUTURE SCOPE

- Integration with mobile application
- Cloud-based monitoring system
- Advanced location analytics
- Power-efficient wearable design